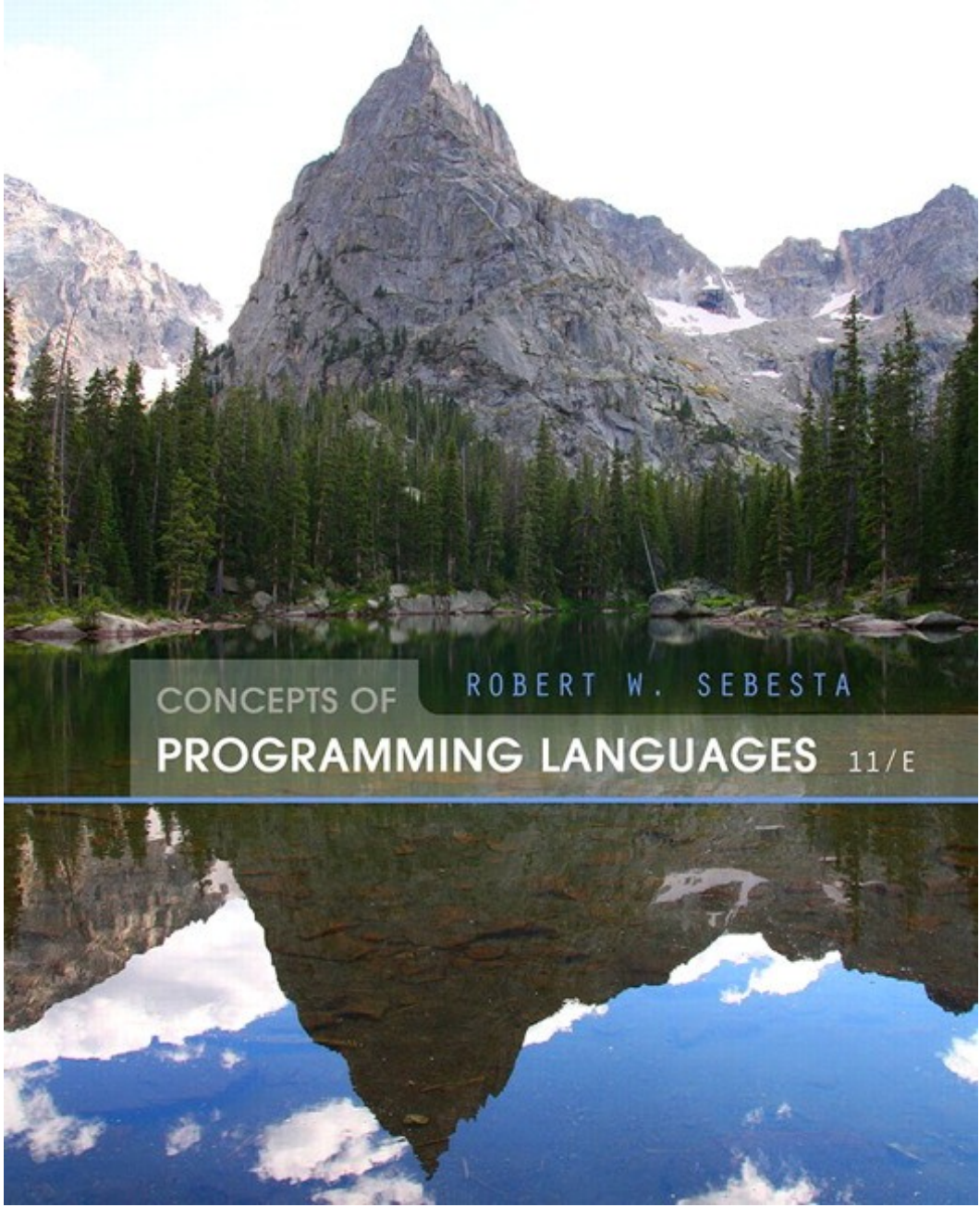
Bölüm 7

İfadeler ve Atama Deyimleri (Expressions and Assignment Statements)



ISBN 0-321-49362-1

Bölüm 7 Konular

- 7.1 Giriş
- 7.2 Aritmetik İfadeler (Arithmetic Expressions)
- 7.3 Aşırı Yüklenmiş Operatörler (Overloaded Operators)
- 7.4 Tip Dönüşümleri (Type Conversions)
- 7.5 İlişkisel ve Boole İfadeleri (Relational and Boolean Expressions)
- 7.6 Kısa Devre Değerlendirmesi (Short-Circuit Evaluation)
- 7.7 Atama İfadeleri (Assignment Statements)
- 7.8 Karışık Mod Atama (Mixed-Mode Assignment)

Copyright © 2015 Pearson. All rights reserved.

7.1 Giriş

- İfadeler (Expressions), bir programlama dilinde hesaplamaları belirtmenin temel araçlarıdır.
- İfade değerlendirmesini anlamak için, operatör ve operand değerlendirmesinin sırasına aşina olmak gerekir.
- İmperatif dillerin özü, atama ifadelerinin (assignment statements) baskın rolüdür.

Copyright © 2015 Pearson. All rights reserved.

7.2 Aritmetik İfadeler (Arithmetic Expressions)

- Aritmetik değerlendirme, ilk programlama dillerinin geliştirilmesindeki motivasyonlardan biriydi.
- Aritmetik ifadeler, operatörler, operandlar, parantezler ve fonksiyon çağrılarıdan oluşur.

Copyright © 2015 Pearson. All rights reserved.

7.2 Aritmetik İfadeler: Tasarım Sorunları

- Aritmetik ifadelerle ilgili tasarım sorunları
  - Operatör önceliği kuralları?
  - Operatör birleşiklik kuralları?
  - Operand değerlendirme sırası?
  - Operand değerlendirmesinin yan etkileri?
  - Operatör aşırı yüklemesi? (Operator overloading?)
  - İfadelerde tür karıştırma?

Copyright © 2015 Pearson. All rights reserved.

7.2 Aritmetik İfadeler: Operators

- Tekli bir operatörün (unary operator) bir operandı vardır.
- İkili bir operatörün (binary operator) iki operandı vardır.
- Üçlü operatör (ternary operator) üç operandı vardır.

Copyright © 2015 Pearson. All rights reserved.

7.2.1 Aritmetik İfadeler: Operatör Öncelik Kuralları

- İfade değerlendirme için *operatör öncelik kuralları (operator precedence rules)*, farklı öncelik düzeylerine sahip “ komşu” operatörlerin değerlendirilme sırasını tanımlar.
- Tipik öncelik düzeyleri
  - parantezler
  - tekli (unary) operatörler
  - ** (dil destekliyorsa)
  - *, /
  - +, -

Copyright © 2015 Pearson. All rights reserved.

7.2.2 Aritmetik İfadeler: Operatör Birleştirilebilirlik Kuralı

- İfade değerlendirme için *operatör birleştirme kuralları (operator associativity rules)*, aynı öncelik düzeyine sahip komşu operatörlerin değerlendirilme sırasını tanımlar.
- Tipik birleştirilebilirlik kuralları
  - Soldan sağa, ** hariç (** sağdan sola)
  - Bazen tekli (unary) operatörler sağdan sola doğru ilişkilendirilir (örneğin, FORTRAN'da).
- APL farklıdır; tüm operatörler eşit önceliğe sahiptir ve tüm operatörler sağdan sola doğru ilişkilendirilir.
- Öncelik (precedence) ve birleştirme (associativity) kuralları parantezlerle geçersiz kılınabilir.

Copyright © 2015 Pearson. All rights reserved.

7.2.3 Ruby ve Scheme'deki ifadeler

- Ruby
  - Tüm aritmetik, ilişkisel ve atama operatörleri ile dizi indeksleme, kaydırma ve bitwise lojik operatörleri metotlar olarak uygulanır.

  - Bunun bir sonucu olarak, bu operatörlerin tümü uygulama programları tarafından geçersiz kılınabilir.
- Scheme (ve Common Lisp)
  - Tüm aritmetik ve lojik işlemler açıkça çağrılan alt programlar tarafından gerçekleştirilir.
  - `a + b * c, (+ a (* b c)) olarak kodlanır`

Copyright © 2015 Pearson. All rights reserved.

7.2.4 Aritmetik İfadeler: Koşullu İfadeler

- Koşullu İfadeler
  - C tabanlı diller (örneğin, C, C++)
  - Bir örnek:

```
average = (count == 0)? 0 : sum / count
```

  - Aşağıdaki gibi yazılmış gibi değerlendirilir:

```
   if (count == 0)
average = 0
   else
average = sum /count
```

Copyright © 2015 Pearson. All rights reserved.

7.2.5 Aritmetik İfadeler: Operand Değerlendirme Sırası

- *Operand Değerlendirme Sırası (Operand Evaluation Order)*
  1. Değişkenler (Variables): bellekten değeri getirir.
  2. Sabitler (Constants): bazen bellekten (memory) alınır; bazen sabit makine dili komutunda (machine language instruction) bulunur.
  3. Parantez içindeki ifadeler (Parenthesized expressions): Önce tüm operandları ve operatörleri değerlendirir.
  4. En ilginç durum, bir operandın bir fonksiyon çağrısı olmasıdır.

Copyright © 2015 Pearson. All rights reserved.

7.2.6 Aritmetik İfadeler: Yan Etki Olasılıkları

- *Fonksiyonel yan etkiler (Functional side effects):* Bir fonksiyon iki yönlü bir parametreyi veya yerel olmayan bir değişkeni değiştirdiğinde
- Fonksiyonel yan etkilerle ilgili sorun:
  - Bir ifadede referans verilen bir fonksiyon, ifadenin başka bir operandını değiştirirse; örneğin, bir parametre değişikliği için.:

```
a = 10;
/* fun parametresini değiştirir varsayalım */
b = a + fun(&a);
```

Copyright © 2015 Pearson. All rights reserved.

7.2.6.1 Fonksiyonel Yan Etkiler

- Soruna karşı iki olası çözüm vardır:
  1. Fonksiyonel yan etkileri yasaklamak için dil tanımını yazın
     - Fonksiyonlarda iki yönlü parametreler yok
     - Fonksiyonlarda yerel olmayan referanslar yok
     - **Avantaj:** İşe yarıyor!
     - **Dezavantaj:** Tek yönlü parametrelerin esnek olmaması ve yerel olmayan referansların eksikliği
  2. Operand değerlendirme sırasının sabitlenmesini talep etmek için dil tanımını yazın.
     - **Dezavantaj**: Bazı derleyici optimizasyonlarını sınırlar
     - Java, operandların soldan sağa doğru sırayla değerlendirilmesini gerektirir.

Copyright © 2015 Pearson. All rights reserved.

7.2.6.2 Referans Şeffaflığı

- Bir program, programdaki aynı değere sahip herhangi iki ifade, programın işleyişini etkilemeden programın herhangi bir yerinde birbirinin yerine kullanılabiliyorsa, *referans şeffaflığı (referential transparency)* özelliğine sahiptir.

```
result1 = (fun(a) + b) / (fun(a) – c);
temp = fun(a);
result2 = (temp + b) / (temp – c);
```

Eğer `fun`'ın yan etkisi yoksa, `result1 = result2` olur.

Aksi takdirde, referans şeffaflığı ihlal edilir.

Copyright © 2015 Pearson. All rights reserved.

7.2.6.2 Referans Şeffaflığı (Devam Ediyor)

- Referans şeffaflığının avantajı
  - Bir programın semantiği, referans şeffaflığı varsa çok daha kolay anlaşılır.
- Değişkenleri olmadığı için, saf fonksiyonel dillerdeki programlar referans açısından şeffaftır.
  - Fonksiyonlar, yerel değişkenlerde depolanacak durumlara sahip olamazlar.
  - Bir fonksiyon dışsal bir değer kullanıyorsa, bu değer sabit olmalıdır (değişkenler yoktur). Dolayısıyla, bir fonksiyonun değeri yalnızca parametrelerine bağlıdır.

Copyright © 2015 Pearson. All rights reserved.

7.3 Aşırı Yüklenmiş Operatörler (Overloaded Operators)

- Bir operatörün birden fazla amaçla kullanılmasına *operatör aşırı yükleme (operator overloading)* denir.
- Bazıları yaygındır (örneğin, **`int`** ve **`float`** için +).
- Bazıları potansiyel sorun teşkil edebilir (örneğin, * C ve C++'da).
  - Derleyici hata algılama kaybı (bir operandın eksikliği algılanabilir bir hata olmalıdır)
  - Okunabilirlikte bir miktar kayıp

Copyright © 2015 Pearson. All rights reserved.

7.3 Aşırı Yüklenmiş Operatörler (Overloaded Operators) (Devam Ediyor)

- C++, C# ve F#, kullanıcı tanımlı aşırı yüklenmiş operatörlere izin verir.
  - Akıllıca kullanıldığında, bu tür operatörler okunabilirliği artırabilir (metot çağrılarını önler, ifadeler doğal görünür).
  - Olası sorunlar:
    - Kullanıcılar anlamsız operasyonlar tanımlayabilir.
    - Operatörler mantıklı olsa bile okunabilirlik olumsuz etkilenebilir.

Copyright © 2015 Pearson. All rights reserved.

7.4 Tip Dönüşümleri (Type Conversions)

- *Daraltıcı dönüştürme (narrowing conversion)*, bir nesneyi orijinal türün tüm değerlerini içeremeyen bir türe dönüştüren dönüştürmedir, örneğin `float`'tan `int`'e.
- *Genişletilmiş dönüştürme (widening conversion)*, bir nesnenin, orijinal türün tüm değerlerine en azından yaklaşık değerler içerebilen bir türe dönüştürülmesidir, örneğin `int`'ten `float`'a.

Copyright © 2015 Pearson. All rights reserved.

7.4.1 Tip Dönüşümleri: Karışık Mod

- *Karışık modlu ifade (mixed-mode expression)*, farklı türlerdeki operandları olan ifadedir.
- *Coercion*, örtük bir tür dönüşümüdür (implicit type conversion).
- Coercion'ların dezavantajı:
  - Derleyicinin tip hatası algılama yeteneğini azaltırlar.
- Çoğu dilde, tüm sayısal türler genişletme dönüşümleri kullanılarak ifadelerde coerce edilir.
- ML ve F#'da ifadelerde coercions yoktur.

Copyright © 2015 Pearson. All rights reserved.

7.4.2 Açık Tür Dönüşümleri (Explicit Type Conversions)

- C tabanlı dillerde *casting* olarak adlandırılır
- Örnekler
  - `C: (int)angle`
  - `F#: float(sum)`

F#'ın syntax'ının fonksiyon çağrılarının syntax'ına benzer olduğunu unutmayın.

Copyright © 2015 Pearson. All rights reserved.

7.4.3 İfadelerdeki Hatalar (Errors in Expressions)

- Nedenler
  - Aritmetiğin doğasında var olan sınırlamalar örneğin, sıfırla bölme (division by zero)
  - Bilgisayar aritmetiğinin sınırlamaları örneğin, overflow
- Çalışma zamanı sistemi (run-time system) tarafından sıklıkla göz ardı edilir

Copyright © 2015 Pearson. All rights reserved.

7.5 İlişkisel ve Boole İfadeleri (Relational and Boolean Expressions)

- İlişkisel İfadeler (Relational Expressions)
  - Çeşitli türlerdeki ilişkisel operatörleri ve operandları kullanır
  - Bazı Boole temsiline göre değerlendirir
  - Kullanılan operatör sembolleri diller arasında biraz farklılık gösterir (`!=`, `/=`, `~=`, `.NE.`, `<>`, `#`)
- JavaScript ve PHP'de iki ek ilişkisel operatör vardır: `===` ve `!==`
  - Kuzenleri ile benzerdir, `==` and `!= ‘te`, operandlarını coerce etmezler.
  - Ruby, coercions kullanan eşitlik ilişkisi operatörü için `==` kullanır ve bunları kullanmayanlar için `eql?` kullanımına izin verir.

Copyright © 2015 Pearson. All rights reserved.

7.5 İlişkisel ve Boole İfadeleri (Relational and Boolean Expressions)

- Boole İfadeleri (Boolean Expressions)
  - Operandlar Boole değerleridir ve sonuç Boole değeridir.
  - Örnek olarak operatörler verilebilir
- C89'da Boolean türü yoktur; false için 0, true için sıfırdan farklı bir değer olan `int` türü kullanılır.
- C dilinin ifadelerinin tuhaf bir özelliği: `a < b < c` geçerli bir ifadedir, ancak sonuç beklediğiniz gibi olmayabilir:
  - Sol operatör değerlendirilir ve 0 veya 1 sonucunu verir.
  - Değerlendirme sonucu daha sonra üçüncü operand (yani **c**) ile karşılaştırılır.

Copyright © 2015 Pearson. All rights reserved.

7.6 Kısa Devre Değerlendirmesi (Short-Circuit Evaluation)

- Sonuc. tüm operandlar ve/veya operatörler değerlendirilmeden belirlenen bir ifade.
- Örnek: `(13 * a) * (b / 13 – 1)`
  a sıfırsa, `(b /13 – 1)` değerini hesaplamaya gerek yoktur
- Kısa devre olmayan değerlendirmeyle ilgili sorun

```
index = 0;
while (index <= length) && (LIST[index] != value)
      index++;
```

  - `index=length` olduğunda, `LIST[index]` bir indeksleme sorununa neden olur (LIST'in uzunluğunun length - 1 olduğunu varsayarsak)

Copyright © 2015 Pearson. All rights reserved.

7.6 Kısa Devre Değerlendirmesi (Short-Circuit Evaluation) (Devam Ediyor)

- C, C++ ve Java: normal Boole operatörleri (&& and ||) için kısa devre değerlendirmesini kullanır, ancak kısa devre olmayan bitwise Boole operatörleri de sağlar (& and |).
- Ruby, Perl, ML, F# ve Python'daki tüm lojik operatörler kısa devre değerlendirilir.
- Kısa devre değerlendirmesi, ifadelerde yan etkilerin potansiyel sorununu ortaya çıkarır.
  Örneğin, `(a > b) || (b++ / 3)`

Copyright © 2015 Pearson. All rights reserved.

7.7 Atama İfadeleri (Assignment Statements)

- Genel syntax

  `<target_var> <assign_operator> <expression>`

- Atama operatörü

  = Fortran, BASIC, the C-based languages

  := Ada

- = eşitlik ilişkisel operatörü için aşırı yüklendiğinde kötü olabilir (bu nedenle C tabanlı diller ilişkisel operatör olarak == kullanır)

Copyright © 2015 Pearson. All rights reserved.

7.7.1 Atama İfadeleri: Koşullu Hedefler

- Koşullu Hedefler (Conditional Targets) (Perl)

```
($flag ? $total : $subtotal) = 0
```

Bu, aşağıdakine eş değerdir

```
if ($flag){
  $total = 0
} else {
  $subtotal = 0
}
```

Copyright © 2015 Pearson. All rights reserved.

7.7.2 Atama İfadeleri: Bileşik Atama Operatörleri (Compound Assignment Operators)

- Sıkça ihtiyaç duyulan bir atama biçimini belirtmenin kısa yolu
- ALGOL'da tanıtıldı; C ve C tabanlı diller tarafından benimsenmiştir.
  - Örneğin

```
a = a + b
```

şöyle yazılabilir

```
a += b
```

Copyright © 2015 Pearson. All rights reserved.

7.7.3 Atama İfadeleri: Tekli Atama Operatörleri (Unary Assignment Operators)

- C tabanlı dillerdeki tekli atama operatörleri, artma ve azalma işlemlerini atama ile birleştirir.
- Örnekler:
  - `sum = ++count` (`count` artırıldı, ardından `sum`'a atandı)
  - `sum = count++` (`count` `sum`'a atandı, ardından artırıldı
  - `count++` (`count` artırıldı)
  - `-count++` (`count` artırıldı, sonra tersine çevrildi)

Copyright © 2015 Pearson. All rights reserved.

7.7.4 İfade Olarak Atama (Assignment as an Expression)

- C tabanlı dillerde, Perl ve JavaScript'te, atama ifadesi bir sonuç üretir ve bir operand olarak kullanılabilir.

```
while ((ch = getchar())!= EOF){…}
```

`ch = getchar()` gerçekleştirilir; sonuç (`ch`'ye atanan) `while` ifadesinin koşul değeri olarak kullanılır.

- Dezavantaj: başka bir tür ifade yan etkisi (expression side effect) oluşur.

Copyright © 2015 Pearson. All rights reserved.

7.7.5 Multiple Assignments

- Perl, Ruby ve Lua, çoklu hedefli çoklu kaynak atamalarına izin verir.

```
($first, $second, $third) = (20, 30, 40);
```

Ayrıca, aşağıdakiler yasaldır ve bir değişim gerçekleştirir:

```
($first, $second) = ($second, $first);
```

Copyright © 2015 Pearson. All rights reserved.

7.7.6 Fonksiyonel Dillerde Atama

- Fonksiyonel dillerde tanımlayıcılar yalnızca değerlerin isimleridir.
- ML
  - İsimler, **`val`** ile değerlere bağlanır.

    ```
    val fruit = apples + oranges;
    ```

  - Eğer başka bir **`val`** `fruit` gelirse, bu yeni ve farklı bir isimdir.
- F#
  - F#'daki **`let`**, ML'deki **`val`** gibidir, ancak **`let`** ayrıca yeni bir scope oluşturur.

Copyright © 2015 Pearson. All rights reserved.

7.8 Karışık Mod Atama (Mixed-Mode Assignment)

- Atama ifadeleri de karışık modda olabilir.
- Fortran, C, Perl ve C++'da, herhangi bir sayısal türdeki değer, herhangi bir sayısal türdeki değişkene atanabilir.
- Java ve C#'da, yalnızca genişletici atama coercions yapılır.
- Ada'da atama coercion'ı yoktur.

Copyright © 2015 Pearson. All rights reserved.

Özet

- İfadeler (Expressions)
- Operatör önceliği ve birleşim özelliği
- Operatör aşırı yükleme (Operator overloading)
- Karışık tip ifadeler (Mixed-type expressions)
- Çeşitli atama türleri

Copyright © 2015 Pearson. All rights reserved.